Informatik I
Dr. Henrik Brosenne
Paul Sarnighausen-Cahn

Wintersemester 2025/26
Georg-August-Universität Göttingen
Institut für Informatik

Übungsblatt 03

Allgemeine Information zum Bestehen der Prüfungsvorleistung.
Um die Prüfungsvorleistung zu bestehen, müssen Sie auf jedem Übungszettel mindestens 40% der Punkte in Theorie und Praxis erreichen. Es gibt jedoch zwei Ausnahmen: Sie dürfen auf zwei Zetteln bei den theoretischen Aufgaben und auf zwei Zetteln beim praktischen Teil unter die 40%-Grenze fallen, ohne dass dies zum Nichtbestehen führt.
Das bedeutet konkret: Wenn Sie beispielsweise auf einem Übungsblatt weniger als 40 % in der Theorie erreichen, können Sie den praktischen Teil trotzdem bestehen – und umgekehrt.

Theoretische Übung

Abgabe bis Di., 25.11., 14 Uhr

Allgemein

Die Lösung dieser Aufgaben werden über das Lernmodul Vips abgegeben.

Formatieren Sie Ihre Abgabe so, dass die Aufgaben 1 und 2 klar durch einzelne Überschriften voneinander getrennt sind.

Aufgabe 1 – 24 Punkte

Zahlensysteme (i1-u-hp0)

Versuchen Sie folgende Aufgaben alleine und ohne Umrechner, z.B. aus dem Internet, zu bearbeiten um ein Gefühl für die Zahlensysteme zu bekommen.

Gegeben sei die Zahl $(83965)_{10}$, rechnen Sie diese Zahl in folgende Zahlensysteme um.

1. Binärsystem
   (4 Punkte)
2. Oktalsystem
   (4 Punkte)
3. Hexadezimalsystem
   (4 Punkte)

Gegeben sei die Zahl $(11011111)_2$, rechnen Sie diese Zahl in folgende Zahlensysteme um.

1. Dezimalsystem
   (4 Punkte)
2. Oktalsystem
   (4 Punkte)
3. Hexadezimalsystem
   (4 Punkte)

Geben Sie jeweils nicht nur das Ergebnis, sondern auch die wesentlichen Teile des **Rechenwegs** mit an.

Aufgabe 2 – 26 Punkte

Codierung von Sprachen (i1-u-aa0)

1. In den 50er Jahren werden bei Ausgrabungen Listen auf Tontafeln gefunden. Die dabei verwendeten Glyphen (Schriftzeichen) sind bis dato nicht bekannt. Die Listen sollen mit einem Computersystem verarbeitet werden, dazu müssen die Glyphen binär codiert werden.

   Aufgrund der Beschränkungen der damaligen Computersysteme, wird eine möglichst effiziente Codierung gesucht.

   a) Die Schrift hat 22 Glyphen (Schriftzeichen).
   $\alpha\ \beta\ \gamma\ \delta\ \epsilon\ \eta\ \chi\ \iota\ \kappa\ \lambda\ \mu\ \nu\ \zeta\ \omega\ \phi\ \pi\ \psi\ \rho\ \sigma\ \tau\ \theta\ \Delta$

   Hinweis. Die Asciimath-Kürzel für diese Glyphen finden sie auf *asciimath.org* unter *Greek Letters* und *Operation symbols*, falls sie diese in ihrer Abgabe nutzen wollen.

   Wie viele Bits sind erforderlich, um die Glyphen einzeln binär zu codieren?
   (2 Punkte)

   b) Bei der Untersuchung der Listen fällt dem Archäologen Dr. Henry Jones Jr. auf, dass alle Einträge eine gerade Anzahl von Glyphen haben. Deshalb schlägt er vor, nicht einzelne Glyphen, sondern Paare von Glyphen binär zu codieren.
   Beispiel Paare: $\alpha\delta\ \beta\rho\ \iota\kappa\ \eta\gamma\ \omega\kappa\ \iota\tau\ \psi\psi$

   Wie viele Bits sind erforderlich für die Glyphenpaare?
   (3 Punkte)

   c) Zeigen Sie, warum die Codierung der Listeneinträge mit der Methode aus Aufgabenteil 1b) tatsächlich eine Verbesserung gegenüber der Methode aus Aufgabenteil 1a) ist.
   (5 Punkte)

2. Die Verarbeitung der Listen mit dem Computersystem hat Erfolg, es kann nachgewiesen werden, dass es sich um Königslisten einer untergegangenen Hochkultur handelt.

   Einige Jahre nach Codierung der Königslisten werden weitere Texte gefunden. Diesmal finden, zusätzlich zu den 22 bekannten Glyphen, weitere 4 Glyphen Verwendung.

   Der erneut hinzugezogene Dr. Jones macht folgende Entdeckungen.

   - Die 4 neuen Glyphen sind Zahlzeichen.
     $\cap\ \cup\ \wedge\ \vee$
   - Eine der 22 Glyphen, die Δ-Glyphe, hat eine spezielle Bedeutung als Abschluss-Glyphe, das war bei den Königslisten noch nicht aufgefallen. Alle Worte oder Zahlen enden mit der Δ-Glyphe, die ausschließlich als letzte Glyphe eines Wortes oder einer Zahl vorkommt.
     $\alpha\delta\ \cancel{\Delta}\beta\ \rho\Delta\ \cup\Delta\forall\cap$

- Worte bestehen nur aus Schriftzeichen. Zahlen bestehen, mit Ausnahme des letzten Zeichens (Δ-Glyphe), nur aus Zahlzeichen.
  $\alpha\delta\ \beta\Delta\ \cup\vee\vee\cap\cup\Delta\ \iota\kappa\ \eta\gamma\ \omega\kappa\ \iota\tau\ \rho\Delta$
- Jedes Wort besteht, inklusive der Δ-Glyphe, aus einer geraden Anzahl von Glyphen.
  $\alpha\delta\ \beta\Delta\ \iota\kappa\ \eta\gamma\ \omega\kappa\ \iota\tau\ \rho\Delta$
- Die Anzahl der Zahlzeichen in einer Zahl unterliegt keiner erkennbaren Gesetzmäßigkeit.
  $\vee\Delta\ \cup\cap\cup\Delta\ \wedge\vee\Delta\ \cap\cap\wedge\vee\cup\wedge\Delta$
- Außer der Δ-Glyphe gibt es keine weiteren Wort-/Zahltrenner, Satzzeichen oder Ähnliches.

Die bei den Königslisten erfolgreich verwendete Codierung von Glyphenpaaren soll auch bei den neu entdeckten Texten angewendet werden.

a) Wie viele Bits sind erforderlich, um alle Glyphen einzeln binär zu codieren?
(2 Punkte)

b) Wie viele Bits sind erforderlich, wenn man Schriftzeichen paarweise und die Zahlzeichen einzeln binär codiert?
(3 Punkte)

Hinweis. Vergessen Sie nicht, dass Zahlen mit der Δ-Glyphe enden.

c) Zeigen Sie, dass die Codierung eines Textes mit der Methode aus Aufgabenteil 2b) nicht in jedem Fall besser ist als die Codierung mit der Methode aus Aufgabenteil 2a).
(3 Punkte)

Hinweis. Geben Sie ein Gegenbeispiel an.

d) Verbessern Sie die Methode aus Aufgabenteil 2b), indem Sie keine einzelnen Zahlzeichen mehr codieren, sondern Paare und Tripel.
(4 Punkte)

e) Zeigen Sie, dass die Codierung von Texten mit mindestens 2 Glyphen mit der Methode aus Aufgabenteil 2d) tatsächlich eine Verbesserung gegenüber der Methode aus Aufgabenteil 2a) ist.
(4 Punkte)

Praktische Übung

Abgabe der Prüfsumme bis Di., 25.11., 14 Uhr
Testat ab Di., 25.11., 18 Uhr

Hilfe zum Bearbeiten der praktischen Übungen können Sie grundsätzlich jeden Tag in den Rechnerübungen bekommen.

- Bearbeiten Sie die praktischen Übungen in dem bereitgestellten Jupyter Notebook.
- Um die praktische Übung testieren zu lassen, **müssen** Sie einen Termin in der zugehörigen Rechnerübung reservieren. Ein Testat ohne Termin ist nicht möglich.
- Übermitteln Sie die Prüfsumme über Vips
- Öffnen Sie die Terminvergabe für diese praktische Übung und lassen Sie den von Ihnen reservierten Termin anzeigen.
- Zeigen Sie ihr Notebook und führen Sie alle zellen aus, damit am Ende die Prüfsumme angezeigt wird.
- Der CIP-Pool befindet sich hier: `https://lageplan.uni-goettingen.de/?ident=2412_1_1.UG_-1.110`

Aufgabe 1 – 15 Punkte

Umwandlung in die Binärdarstellung (i1-p-ha0)

In der Vorlesung wurde gezeigt, wie man eigene Funktionen in Python definieren kann (zum Beispiel die Funktionen `gauss_formula(n)` oder `gcd(a, b)` im Skript). Eigene Funktionen dienen dazu, Berechnungen zu strukturieren und Code wiederzuverwenden.
In dieser Aufgabe sollen Sie eine Funktion schreiben, die eine natürliche Zahl in ihre Darstellung zur Basis 2 (Binärdarstellung) umwandelt. Verwenden Sie ausschließlich grundlegende Sprachmittel aus dem Abschnitt *„Ablaufsteuerung und Arithmetik in Python“*: Variablen, Ganzzahldivision `//`, Modulo `%`, Typumwandlung.

Aufgabe: Vervollständigen Sie im bereitgestellten Juypter Notebook folgende Funktion:

```
    # Sonderfall: n = 0
    if n == 0:
        return "0"

    result = ""

    # Solange die Zahl noch größer als 0 ist:
    while n > 0:
  # Hier kommt Ihr Code
    return result
```

, die eine natürliche Zahl `n` entgegennimmt und einen String zurückgibt, der die Binärdarstellung von `n` enthält.

Funktionale Anforderungen:

- Eingabe: eine natürliche Zahl `n >= 0`.
- Ausgabe: ein String mit der Binärdarstellung von `n`.
- Die Umwandlung muss arithmetisch erfolgen: Restbestimmung mittels `n % 2` und Ganzzahldivision mittels `n // 2`.
- Sonderfall: Für `n == 0` ist das Ergebnis der String `"0"`.

Testen Sie Ihre Funktion in einer weiteren Notebook-Zelle, z. B. mit:

```
print(to_binary(0), '?=', bin(0))
print(to_binary(1), '?=', bin(1))
print(to_binary(19), '?=', bin(19))
print(to_binary(255), '?=', bin(255))
```

(15 Punkte)

Aufgabe 2 – 20 Punkte

Middle Square Generator (i1-p-pq0)

Eine Folge mit einfachem rekursiven Bildungsgesetz ist einen Auflistung von (unendlich vielen) fortlaufend nummerierten Folgengliedern (Zahlen), wobei ein Folgenglied (nur) aus dem unmittelbar vorhergehenden Folgenglied berechnet wird. Das erste Folgenglied (Startwert) wird explizit angegeben, somit können mit demselben Bildungsgesetz verschiedene Folgen aus unterschiedlichen Startwerten erzeugt werden.
In dieser Aufgabe sollen Sie einen einfachen Middle-Square-Generator für zweistellige Zahlen implementieren. Dabei dürfen Sie ausschließlich grundlegende Sprachmittel verwenden, wie sie im Abschnitt *„Ablaufsteuerung und Arithmetik in Python“* des Skripts behandelt wurden: Variablen, Ganzzahldivision (`//`), Modulo (`%`), `while`-Schleifen, bedingte Anweisungen. .
Wichtig: Es dürfen keine `import`-Anweisungen verwendet werden.

Hinweis zu Eingaben:

- Die Funktion `input()` liest eine Zeichenkette von der Tastatur ein, siehe `https://docs.python.org/3/library/functions.html#input`.
- Die Funktion `int()` wandelt eine Zeichenkette in eine ganze Zahl um, siehe `https://docs.python.org/3/library/functions.html#int`.
- Sie dürfen das Ergebnis von `input()` als Argument von `int()` verwenden, z. B.: `n = int(input("Startwert: "))`.

Aufgabe:
Schreiben Sie ein Programm, das wie folgt arbeitet:

1. Liest einen Startwert ein und überprüft, ob es sich um eine zweistellige, nichtnegative Zahl handelt. Ist dies nicht der Fall, soll das Programm eine Fehlermeldung ausgeben und beenden.
2. Erzeugt mit Hilfe einer `while`-Schleife die ersten 101 Folgenglieder der Middle-Square-Folge. Jede Iteration soll:
   a) den aktuellen Wert ausgeben,
   b) den Wert quadrieren,
   c) den neuen Wert berechnen, indem zuerst die Einerstelle entfernt wird (`squared // 10`) und anschließend die Tausenderstelle abgeschnitten wird (`% 100`).
3. Kommentieren Sie im Code die Schritte *Ausgabe*, *Quadrieren* und *Aktualisieren*.

(10 Punkte)

Eine Folge kann einen Zyklus, d. h. eine sich wiederholende minimale Teilfolge, enthalten. Eine Folge $f_0, f_1, f_2, \ldots$ enthält einen Zyklus $g_1, \ldots, g_n$ der Länge n, wenn es einen Index $i \geq 0$ gibt, sodass Folgendes gilt:

$$f_0, f_1, f_2, \ldots = f_0, \ldots, f_{i-1}, \underbrace{g_1, \ldots, g_n}_{\text{Zyklus}}, g_1, \ldots, g_n, g_1, \ldots, g_n, \ldots$$

wobei $f_i = g_1$ und $g_1 \neq g_k$ für alle $k = 2, \ldots, n$.
Finden und demonstrieren Sie die Funktionsweise mit verschiedenen Eingabewerten, die folgende Folgen erzeugt:

1. Folge beginnt mit einem Zyklus der Länge 1
2. Folge beginnt mit einem Zyklus der Länge 2.
3. Folge enthält einen Zyklus der Länge 1, beginnt aber nicht mit einem Zyklus.
4. Folge enthält einen Zyklus der Länge 2, beginnt aber nicht mit einem Zyklus.

Gibt es Folgen mit einem Zyklus der Länge größer 2?
(10 Punkte)

Aufgabe 3 – 15 Punkte

Teilbarkeit (i1-p-hn0)

In dieser Aufgabe sollen Sie Zahlen auf ihre Teilbarkeit durch 9, 11 und 29 prüfen und das kleinste gemeinsame Vielfache (kgV) dieser Zahlen ermitteln. **Zur Überprüfung der Teilbarkeit dürfen Sie ausschließlich die unten beschriebenen Regeln und selbstgeschriebene Python-Funktionen verwenden.**

- Eine Zahl ist durch 9 teilbar, wenn ihre **iterierte Quersumme** gleich 9 ist.
- Eine Zahl ist durch 11 teilbar, wenn ihre **iterierte alternierende Quersumme** gleich 0 ist.
  - Bei der alternierenden Quersumme wird jede zweite Ziffer abgezogen, anstatt addiert.
  - Beispiel:
    $$123456 \to 6 - 5 + 4 - 3 + 2 - 1 = 3$$
  - Hinweis:
    - Das Ergebnis der alternierenden Quersumme kann negativ sein.
    - Eine negative Zahl hat dieselben Teiler wie ihr positiver Betrag.
- Eine Zahl ist durch 29 teilbar, wenn gilt:

  $$\text{letzte Ziffer} \times 3 + \text{Rest der Zahl ohne letzte Ziffer}$$

  ist durch 29 teilbar.
  - Diese Regel kann mehrfach angewandt werden, bis sich eine Zahl ergibt, die kleiner oder gleich 29 ist.
  - Beispiel:

    $123456 \to 12345{+}3{\times}6 = 12363 \to 1236{+}3{\times}3 = 1245 \to 124{+}3{\times}5 = 139 \to \ldots$

1. Schreiben Sie eine **Python-Funktion** `check_divisibility(n)`, die eine natürliche Zahl `n` als Parameter erhält und prüft, ob sie durch 9, 11 und 29 teilbar ist. Geben Sie für jede Zahl eine entsprechende Textausgabe aus, z. B.:

   ```
   >>> check_divisibility(594)
   594 ist durch 9 teilbar.
   594 ist nicht durch 11 teilbar.
   594 ist nicht durch 29 teilbar.
   ```

   (10 Punkte)

2. Schreiben Sie auf Grundlage der Funktion aus Aufgabenteil 1 ein Python-Programm, das alle natürlichen Zahlen von 1 bis 10 000 durchläuft und die Zahlen ausgibt, die **gleichzeitig durch 9, 11 und 29 teilbar** sind. Verwenden Sie dafür eine **for-Schleife** und ggf. die Python-Funktion `range()`. (5 Punkte)